

Absolutely. For **Artificial Intelligence (CS24307)**, I'll make it in the same way as DMCT/DCCN: **complete M1–M5 notes, every syllabus point covered, exam-oriented explanations, algorithms, examples, diagrams, formulas, active-recall questions, and PYQ-style questions.**

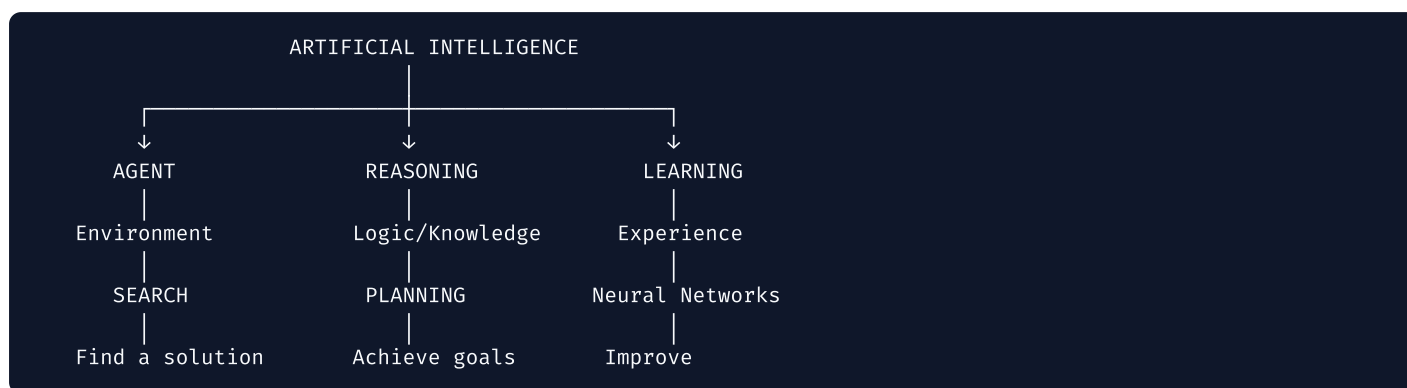
I checked your actual BIT Mesra syllabus rather than using a generic AI syllabus. The official course is **CS24307, Artificial Intelligence, Semester V, 3 credits.**

One important difference from DMCT: **I could not find a separate CS24307 written Mid/End PYQ among your uploaded files**, so I won't falsely label invented questions as actual PYQs. I'll clearly mark questions from your uploaded AI study material as **practice/expected questions**, while incorporating the AI-lab questions where useful. Your AI syllabus itself is very clear about the five modules.

ARTIFICIAL INTELLIGENCE — COMPLETE MASTER NOTES

CS24307 | M1–M5 | 38 Core Topics

The entire subject in one mental model



And the progression is:

Key Insight

Perceive → Represent → Search/Reason → Plan → Learn

MODULE I — PRELIMINARIES

7/7 topics

What is AI?

Evolution of AI

Intelligent Agents

Concept of Rationality

Nature of Environments

Structure of Agents

Applications of AI

1. What is Artificial Intelligence?

Definition

Artificial Intelligence is the field of computer science concerned with building systems capable of performing tasks that require capabilities associated with intelligence, such as:

- reasoning
- learning
- perception
- planning
- problem solving
- decision making
- language understanding

Russell & Norvig perspective

AI can be understood through the idea of an **intelligent agent**:

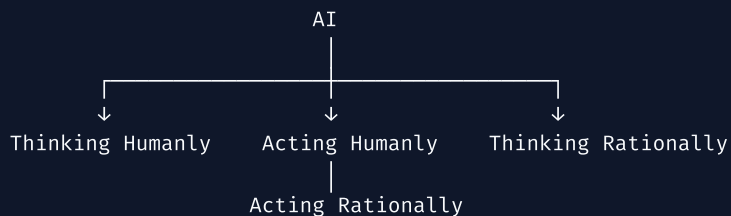
Key Insight

An agent perceives its environment and takes actions to achieve goals.

John McCarthy

AI is concerned with making intelligent machines.

Four classical approaches to AI



Thinking humanly

Model human thought processes.

Acting humanly

Make machines behave like humans.

Thinking rationally

Use formal logic to derive correct conclusions.

Acting rationally

Choose actions that maximize expected performance.

Most important modern perspective

Rational agent approach.

2. Evolution of AI

Timeline

```
1950
↓
Turing Test
↓
1956
Dartmouth + "Artificial Intelligence"
↓
1960-70
Symbolic AI
↓
1980s
Expert Systems
↓
1990s
Machine Learning
↓
1997
Deep Blue
↓
2011
Watson
↓
2016
AlphaGo
↓
2020+
Deep Learning + Generative AI
```

Important milestones

1950 — Alan Turing

Proposed the **Turing Test**.

1956 — Dartmouth Conference

John McCarthy popularized the term **Artificial Intelligence**.

1980s — Expert systems

Rule-based systems became commercially important.

1997 — Deep Blue

IBM's chess system defeated Garry Kasparov.

2011 — Watson

IBM Watson defeated champions on *Jeopardy!*.

2016 — AlphaGo

Defeated Lee Sedol in Go.

2020s

Large-scale deep learning and generative AI became dominant areas.

Your uploaded AI study material uses essentially this timeline.

Key Insight

T → D → E → ML → DL → GenAI

Turing → Dartmouth → Expert Systems → Machine Learning → Deep Learning → Generative AI

3. Intelligent Agents

Agent

An **agent** perceives its environment through sensors and acts upon it through actuators.



Agent loop

Key Insight

Sense → Think → Act

Examples

Agent	Sensors	Actuators
Robot vacuum	Dirt/bump sensors	Wheels, suction
Self-driving car	Camera, LiDAR, GPS, radar	Steering, brakes
Thermostat	Temperature sensor	Heater/AC
Chess program	Board state	Move
Voice assistant	Microphone	Speaker

4. Rationality

A **rational agent** chooses the action expected to maximize its performance measure, given:

percept sequence

knowledge

available actions

environment

Rational ≠ omniscient

This is extremely important.

A rational agent does **not** necessarily know the future.

It chooses the best action based on available information.

Rational-agent framework

```
Percept history
+
Knowledge
+
Possible actions
↓
Rational decision
↓
Action
```

5. Nature of Environments

Environment characteristics:

1. Fully observable vs Partially observable

Fully observable: sensors provide complete relevant state.

Example: Chess.

Partially observable: some state information is hidden.

Example: Poker.

2. Deterministic vs Stochastic

Deterministic

Current state + action completely determines next state.

Stochastic

Randomness/uncertainty affects outcome.

Example:

```
Chess → deterministic
Weather → stochastic
```

3. Episodic vs Sequential

Episodic

Each decision is independent.

Example: image classification.

Sequential

Current action affects future states.

Example: chess.

4. Static vs Dynamic

Static

Environment does not change while agent deliberates.

Dynamic

Environment can change.

Example:

```
Crossword → static
Driving → dynamic
```

5. Discrete vs Continuous

Discrete

Finite/countable states/actions.

Example: chess.

Continuous

Values vary continuously.

Example: autonomous driving.

6. Single-agent vs Multi-agent

Single agent

Only one agent significantly acts.

Multi-agent

Multiple agents interact.

Example:

```
Sudoku → single
Traffic → multi
```

Memory

Key Insight

O-D-E-D-C-N

Observable

Deterministic

Episodic

Dynamic

Continuous

Number of agents

This exact memory aid appears in your uploaded Module-1 material.

6. Structure of Agents

General architecture:



Agent function

Mathematical mapping:

\$\$

$f:P^*\rightarrow A$

\$\$

where:

$\backslash(P^*)$ = sequence of percepts

$\backslash(A)$ = actions

Agent function vs Agent program

Agent Function	Agent Program
Abstract	Concrete
Maps percepts → actions	Implements mapping
Mathematical concept	Software

Types of Intelligent Agents

1. Simple Reflex Agent

Uses current percept.

```
IF condition
THEN action
```

Example:

Key Insight

If temperature $< 20^{\circ}\text{C}$ $\rightarrow$ heater ON.

Limitation

Cannot handle situations requiring memory.

2. Model-Based Reflex Agent

Maintains internal state.

```
Percept
↓
Internal State
↓
Rule
↓
Action
```

Useful when environment is partially observable.

3. Goal-Based Agent

Has explicit goals.

```
Current State
↓
Goal
↓
Search / Planning
↓
Action
```

Example:

Key Insight

Navigation system finding route to destination.

4. Utility-Based Agent

Uses a utility function to compare possible outcomes.

\$\$

Utility(state)

\$\$

Chooses the action with highest expected utility.

Example:

A self-driving car may balance:

safety

speed

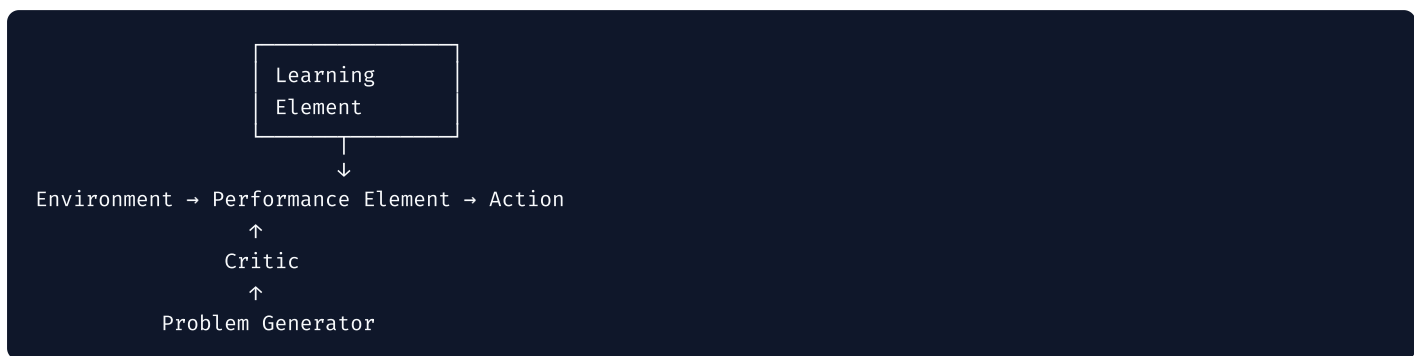
comfort

fuel

5. Learning Agent

Improves using experience.

Four components:



Four components

Performance element

Learning element

Critic

Problem generator

Power progression

Key Insight

Simple Reflex < Model-Based < Goal-Based < Utility-Based < Learning

Your uploaded Module-1 material explicitly emphasizes this progression.

7. Applications of AI

Major domains:

Domain	Examples
Healthcare	Diagnosis, imaging, drug discovery
Finance	Fraud detection, risk analysis
Education	Personalized learning
Transportation	Autonomous vehicles
E-commerce	Recommendation
Cybersecurity	Threat detection
NLP	Translation, chatbots
Computer Vision	Recognition
Robotics	Automation
Games	Chess, Go

M1 ACTIVE RECALL

Close the notes and answer:

What is AI?

What are the four approaches to AI?

What is an intelligent agent?

Sensor vs actuator?

Define rational agent.

Why is rationality different from omniscience?

Explain PEAS.

What are the six environment properties?

Simple reflex vs model-based agent?

Goal-based vs utility-based agent?

Explain learning-agent architecture.

Agent function vs agent program.

Give PEAS for an autonomous car.

Your uploaded AI Module-1 material itself includes 2-, 5-, and 10-mark questions around these areas.

MODULE II — PROBLEM SOLVING BY SEARCH AGENT

6/6 topics

Search-based problem solving

State-space search

Heuristic search

Local search

Search in complex environments

Game-tree search

This is the **algorithm-heavy module**.

8. Search-Based Problem Solving

An agent can solve a problem by searching through possible states.

```
Initial State
  ↓
Possible Actions
  ↓
Successor States
  ↓
Search
  ↓
Goal State
```

Problem formulation

A problem generally consists of:

Initial state

Actions

Transition model

Goal test

Path-cost function

Example: Route finding

```
Initial = Ranchi
Goal = Delhi
Actions = roads
Transition = road movement
Cost = distance/time
```

9. State Space Search

A **state space** is the collection of possible states reachable through actions.

Example:



S = initial state

G = goal
edges = actions

Uninformed Search

Search without domain-specific heuristic knowledge.

Major algorithms:

BFS

DFS

Uniform Cost Search

Depth-Limited Search

Iterative Deepening DFS

Breadth-First Search

Explores shallowest nodes first.

```
Level 0:      S
              / \
Level 1:      A  B
              / \ / \
Level 2:      C D E F
```

Data structure

Queue

FIFO.

Properties

If branching factor = $\sqrt{b}$, shallowest solution depth = $\sqrt{d}$:

Complete: Yes, under standard finite-branching assumptions

Optimal: Yes, when step costs are equal

Time: $\mathcal{O}(b^{d+1})$

Space: $\mathcal{O}(b^{d+1})$

Depth-First Search

Explores deepest available node first.

Data structure

Stack / recursion

Properties

Memory efficient

Not generally optimal

Can get stuck down an infinite path

Complete only under appropriate finite conditions

Approximate complexity:

\$\$

$O(b^m)$

\$\$

where m = maximum depth.

Uniform Cost Search

Expands node with lowest path cost.

\$\$

$g(n)$

\$\$

Priority queue

lowest $g(n)$ first

Important

UCS is optimal when step costs are positive.

BFS vs DFS vs UCS

	BFS	DFS	UCS
Strategy	Shallowest	Deepest	Lowest cost
Structure	Queue	Stack	Priority queue
Optimal	Equal costs	No	Yes
Memory	High	Low	High
Uses cost?	No	No	Yes

10. State-Space Search

Represent the problem explicitly:

\$\$

State $\rightarrow$ Action $\rightarrow$ Successor

\$\$

Search tree vs state-space graph

Search tree

May contain duplicate states.

Search graph

Maintains explored/visited states to avoid repeated work.

Graph search

```
Frontier
+
Explored set
```

prevents unnecessary repeated exploration.

11. Heuristic Search

Uses domain knowledge.

A heuristic:

\$\$

$h(n)$

\$\$

estimates cost from node n to a goal.

Example:

For route finding:

\$\$

$h(n) = \text{straight-line distance to destination}$

\$\$

Greedy Best-First Search

Uses:

\$\$

$f(n) = h(n)$

\$\$

Chooses node that appears closest to goal.

Advantage

Often fast.

Problem

Not guaranteed optimal.

A* Search

Uses:

\$\$

$$f(n) = g(n) + h(n)$$

\$\$

where:

$g(n)$ = cost already incurred

$h(n)$ = estimated remaining cost

Memory trick

Key Insight

A* = Already paid + estimated

Admissible heuristic

A heuristic is admissible if:

\$\$

$$h(n) \leq h^*(n)$$

\$\$

It never overestimates the actual remaining cost.

With appropriate conditions, A* tree search is optimal.

Consistent heuristic

For every edge:

\$\$

$$h(n) \leq c(n, a, n') + h(n')$$

\$\$

Consistency implies admissibility in standard settings.

12. Local Search

Local search focuses on improving the current state rather than constructing a complete path.

Useful for:

optimization

scheduling
N-Queens
travelling salesman variants

Hill Climbing

Move to a better neighboring state.

```
Current
↓
Best neighbor
↓
Best neighbor
↓
...
```

Problems

Local maximum

Better than nearby states but not globally best.

Plateau

Neighbors have similar values.

Ridge

Best direction is difficult to reach through available local moves.

Variants

Simple hill climbing
Steepest-ascent hill climbing
Stochastic hill climbing

Simulated Annealing

Sometimes accepts worse states to escape local maxima.

Acceptance probability conceptually:

\$\$

$$P = e^{-\Delta E / T}$$

\$\$

As temperature (T) decreases:

Key Insight

exploration decreases → convergence increases.

Mental model

Key Insight

Hill climbing = greedy.

Key Insight

Simulated annealing = sometimes take a bad step to escape a trap.

Genetic Algorithms

Population-based optimization.

```
Population
↓
Fitness
↓
Selection
↓
Crossover
↓
Mutation
↓
New Population
```

Although not explicitly named in the 6-item outline, this is useful as a local/optimization concept when studying AI search broadly.

13. Search in Complex Environments

Real environments may be:

partially observable

nondeterministic

dynamic

multi-agent

unknown

Therefore standard search may not be sufficient.

The agent may need:

belief states

contingency plans

online search

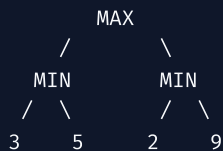
exploration

replanning

14. Game Tree Search

Used for adversarial environments.

Example:



MAX chooses maximum.

MIN chooses minimum.

Minimax

For two-player zero-sum games:

MAX maximizes utility

MIN minimizes utility

Procedure

```
Build tree
↓
Evaluate terminal states
↓
MIN chooses minimum
↓
MAX chooses maximum
↓
Root gives best move
```

Alpha-Beta Pruning

Optimization of minimax.

Two values:

\$\$

$\alpha = \text{best MAX value so far}$

\$\$

\$\$

$\beta = \text{best MIN value so far}$

\$\$

Prune when:

\$\$

$\alpha \geq \beta$

\$\$

Key point

Key Insight

Alpha-beta gives the **same minimax result**, but explores fewer nodes.

Your uploaded AI-lab files specifically include **game-tree generation, Minimax and Alpha-Beta pruning**, making these especially useful for both theory and implementation.

M2 ALGORITHM MEMORY

```
BFS      → Queue → shallow
DFS      → Stack → deep
UCS      → g(n) → cheapest
Greedy   → h(n) → looks closest
A*       → g+h → cost + estimate
Hill     → better neighbor
Annealing → occasionally worse
Minimax  → MAX vs MIN
AlphaBeta → Minimax + pruning
```

MODULE III — KNOWLEDGE REPRESENTATION & REASONING

9/9 topics

Knowledge-based agents
Propositional logic
Propositional → predicate logic
Propositional logic-based agents
First-order predicate logic
Knowledge representation in FOL
Forward chaining
Backward chaining
Resolution

15. Knowledge-Based Agents

A knowledge-based agent contains:

```
Knowledge Base
  ↓
Inference Engine
  ↓
Decision
  ↓
Action
```

Knowledge Base

Stores facts and rules.

Example:

```
Human(Socrates)
Human(x) → Mortal(x)
```

TELL and ASK

A knowledge-based agent commonly uses:

TELL

Add information to KB.

ASK

Query KB.

```
TELL(KB, fact)
ASK(KB, query)
```

16. Propositional Logic

Deals with propositions that are either:

True

False

Examples:

```
P = It is raining
Q = Road is wet
```

Operators

Symbol	Meaning
$\neg P$	NOT
$P \wedge Q$	AND
$P \vee Q$	OR
$P \rightarrow Q$	implication
$P \leftrightarrow Q$	biconditional

Truth Table Basics

For implication:

P	Q	$P \rightarrow Q$
T	T	T
T	F	F
F	T	T
F	F	T

Important:

Key Insight

Implication is false only when P is true and Q is false.

Logical Equivalence

Two statements are equivalent if they have the same truth value in every interpretation.

Important equivalence:

$$P \rightarrow Q \equiv \neg P \vee Q$$

Double negation:

$$\neg(\neg P) \equiv P$$

De Morgan:

$$\neg(P \wedge Q) \equiv \neg P \vee \neg Q$$

$\neg(P \wedge Q) \equiv \neg P \vee \neg Q$

\$\$

\$\$

$\neg(P \vee Q) \equiv \neg P \wedge \neg Q$

\$\$

17. Propositional → Predicate Logic

Propositional logic treats statements as indivisible units.

Predicate logic allows us to represent:

objects

properties

relationships

quantities

Example:

Propositional

P = Ram is a student

Predicate

\$\$

Student(Ram)

\$\$

18. Propositional Logic–Based Agents

Agent uses logical sentences to represent its world.

Classic example:

Key Insight

Wumpus World.

Agent:

```
Percepts
↓
Logical KB
↓
Inference
↓
Action
```

19. First-Order Predicate Logic

FOPL provides:

constants
variables
predicates
functions
quantifiers

Constants

Specific objects.

```
Ram  
Delhi  
Book1
```

Variables

```
x, y, z
```

Predicates

Properties/relations.

```
$$  
Student(x)  
$$  
$$  
Likes(x,y)  
$$  
---
```

Quantifiers

Universal

```
$$  
\forall x  
$$  
means:
```

Key Insight

for every x.

Example:

\$\$

$\forall x(\text{Student}(x) \rightarrow \text{Human}(x))$

\$\$

Existential

\$\$

$\exists x$

\$\$

means:

Key Insight

there exists an x.

Example:

\$\$

$\exists x(\text{Student}(x) \wedge \text{Intelligent}(x))$

\$\$

20. Knowledge Representation in FOL

Example:

Key Insight

Every student studies.

\$\$

$\forall x(\text{Student}(x) \rightarrow \text{Studies}(x))$

\$\$

Key Insight

Sh is a student.

\$\$

$\text{Student}(\text{Sh})$

\$\$

Therefore:

\$\$

$\text{Studies}(\text{Sh})$

\$\$

This is inference.

21. Forward Chaining

Data-driven reasoning.

Start with known facts and repeatedly apply rules.

Example:

Fact:

A

Rule:

$A \rightarrow B$

Rule:

$B \rightarrow C$

Therefore:

A

↓

B

↓

C

Algorithm

Known facts

↓

Find applicable rule

↓

Add conclusion

↓

Repeat

Best mental cue

Key Insight

Forward = Facts $\rightarrow$ Goal

22. Backward Chaining

Goal-driven reasoning.

Start from the goal and ask:

Key Insight

What rules could prove this?

Example:

Goal: C

↑

B → C

↑

A → B

↑

Fact A

Mental cue

Key Insight

Backward = Goal → Facts

Forward vs Backward

Forward	Backward
Data-driven	Goal-driven
Starts facts	Starts query
Derives consequences	Finds supporting rules
Can generate many facts	Focused search

23. Resolution

Resolution is a powerful inference rule used to prove statements by contradiction.

Basic rule:

\$\$

$(P \vee Q), (\neg Q \vee R)$

\$\$

resolve on Q:

\$\$

$P \vee R$

\$\$

Resolution strategy

To prove:

\$\$

$KB \models Q$

\$\$

add:

\$\$

\neg Q

\$\$

to KB and derive contradiction.

```
KB + ¬Q
  ↓
Resolution
  ↓
Empty clause □
  ↓
Q proven
```

Empty clause

\$\$

\Box

\$\$

means contradiction.

M3 MEMORY

```
Propositional → statements
FOL           → objects + relations
Forward       → facts → goal
Backward      → goal → facts
Resolution    → contradiction → proof
```

MODULE IV — PLANNING & PROBABILISTIC REASONING

7/7 topics

Planning in AI

Components of planning problem

Types of planning

Goal Stack Planning

Reasoning under uncertainty

Bayesian inference

Probabilistic reasoning

24. Planning in AI

Planning means finding a sequence of actions that achieves a goal.

```
Initial State
```

```
↓
```

```
Action 1
```

```
↓
```

```
Action 2
```

```
↓
```

```
Action 3
```

```
↓
```

```
Goal
```

Example:

Key Insight

Robot must move a box from A to B.

25. Components of Planning Problem

A planning problem includes:

Initial state

Where we begin.

Goal state

What we want.

Actions/operators

Available actions.

Each action has:

Preconditions

Effects

Example:

```
Action: Move(A,B)
```

```
Precondition:
```

```
At(robot,A)
```

```
Effect:
```

```
At(robot,B)
```

```
-At(robot,A)
```

STRIPS Representation

Common action representation:

```
ACTION
Preconditions
Add effects
Delete effects
```

Example:

```
Move(A,B)

PRE:
At(Robot,A)

ADD:
At(Robot,B)

DELETE:
At(Robot,A)
```

26. Types of Planning

State-space planning

Search through states.

Goal-stack planning

Break goals into subgoals.

Partial-order planning

Only specify ordering constraints that are necessary.

Conditional planning

Handles different possible outcomes.

Hierarchical planning

Breaks high-level tasks into lower-level actions.

27. Goal Stack Planning

Uses a stack of goals/actions.

Suppose:

```
Goal: At(Robot,B)
```

Planner asks:

Key Insight

What action achieves this?

Move(A,B)

Then asks:

Key Insight

What does Move require?

At(Robot,A)

Solve that first.

```
Goal
↓
Action
↓
Preconditions
↓
Subgoals
↓
Actions
```

28. Reasoning Under Uncertainty

Real-world AI rarely has perfect information.

Reasons:

incomplete knowledge

noisy sensors

unpredictable environment

ambiguous observations

Instead of:

\$\$

True/False

\$\$

we may use:

\$\$

P(event)

\$\$

Probability Basics

\$\$

$0 \leq P(A) \leq 1$

\$\$

Complement:

\$\$

$$P(\neg A) = 1 - P(A)$$

\$\$

Joint probability:

\$\$

$$P(A, B)$$

\$\$

Conditional probability:

\$\$

$$P(A|B) = \frac{P(A, B)}{P(B)}$$

\$\$

29. Bayesian Inference

Bayes' theorem:

\$\$

$$P(H|E) =$$

$$\frac{P(E|H)P(H)}{P(E)}$$

where:

\$\$

H = hypothesis

E = evidence

Memory

Key Insight

Posterior = Likelihood × Prior / Evidence

Example

Disease diagnosis:

```
Disease
↓
Probability of symptom
↓
Observed symptom
↓
Update disease probability
```

Bayesian reasoning lets evidence modify our belief.

30. Probabilistic Reasoning

AI can combine uncertain evidence.

Example:

```
Cloudy
+
Weather history
+
Humidity
↓
P(Rain)
```

Bayesian Network

A Bayesian Network is a **directed acyclic graph (DAG)** representing probabilistic dependencies.

Example:

```
Cloudy
 /  \
↓    ↓
Rain  Sprinkler
 \   /
  ↓  ↓
WetGrass
```

Each node has a conditional probability distribution.

Joint probability

For variables:

\$\$

$X_1, X_2, \dots, X_n$

\$\$

a Bayesian network represents:

\$\$

$P(X_1, \dots, X_n)$

=

$\prod_i P(X_i | \text{Parents}(X_i))$

\$\$

This is one of the most important formulas in probabilistic AI.

Bayes vs Bayesian Network

Bayes theorem	Bayesian Network
Formula	Graphical model
Updates probability	Represents dependencies
Can solve individual inference	Supports complex probabilistic reasoning

MODULE V — LEARNING

9/9 topics

What is learning?
Rote learning
Learning by taking advice
Learning from examples
Induction
Formal learning theory
Neural net learning
Underfitting
Overfitting

31. What is Learning?

Learning means improving performance based on experience/data.

A learning system:

```
Experience
  ↓
Learning
  ↓
Knowledge/Model
  ↓
Better performance
```

32. Rote Learning

Simplest form.

Key Insight

Store previous solutions and reuse them.

Example:

```
Problem A → Solution A
```

If Problem A appears again:

```
retrieve Solution A
```

Advantage

Very fast retrieval.

Limitation

Doesn't generalize well to new problems.

33. Learning by Taking Advice

Machine receives knowledge from an external source such as:

teacher

expert

human

documentation

Example:

```
Teacher:  
"Never move the king into check."
```

```
Agent:  
stores rule
```

34. Learning from Examples

Learns a general model from examples.

```
Training Examples  
↓  
Learning Algorithm  
↓  
Hypothesis / Model  
↓  
Prediction
```

Example:

```
Emails + labels  
↓  
Classifier  
↓  
New email → Spam / Not Spam
```

35. Induction

Induction derives general rules from specific examples.

Example:

```
Swan 1 → white
Swan 2 → white
Swan 3 → white
    ↓
Hypothesis:
Swans are white
```

But induction is not logically guaranteed to be universally true.

Deduction vs Induction

Deduction	Induction
General → specific	Specific → general
Conclusion logically follows	Conclusion is probable
Rule application	Rule discovery

36. Formal Learning Theory

Studies the theoretical foundations of learning.

Important concepts:

hypothesis

hypothesis space

training examples

generalization

error

sample complexity

bias

variance

Hypothesis

A candidate model explaining data.

Example:

```
h(x)=classification rule
```

Hypothesis space

Set of all hypotheses the learner can choose from.

$$H = \{h_1, h_2, h_3, \dots\}$$

Learning attempts to select a good hypothesis.

Generalization

A model should perform well on **unseen data**, not merely memorized training data.

```
Training data
  ↓
Learn
  ↓
Model
  ↓
Unseen data
  ↓
Generalization
```

37. Neural Net Learning

Neural networks consist of interconnected neurons.

```
Input Layer
  ↓
Hidden Layer
  ↓
Output Layer
```

A neuron computes:

\$\$

$$z = \sum_i w_i x_i + b$$

\$\$

Then:

\$\$

$$y = f(z)$$

\$\$

where f is an activation function.

Common activation functions

Sigmoid

\$\$

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

\$\$

Range:

\$\$

(0,1)

\$\$

ReLU

\$\$

$$\text{ReLU}(x) = \max(0, x)$$

\$\$

Tanh

\$\$

$$\tanh(x)$$

\$\$

Range:

\$\$

(-1,1)

\$\$

Gradient Descent

Objective:

\$\$

\text{minimize loss}

\$\$

Weight update:

\$\$

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w}$$

\$\$

where:

\(\eta\) = learning rate

\(L\) = loss

Backpropagation

Although the official learning module names **neural-net learning** rather than explicitly saying "backpropagation," backpropagation is the central training mechanism you should understand.

```
Input
↓
Forward pass
↓
Prediction
↓
Loss
↓
Backward pass
↓
Gradients
↓
Update weights
↓
Repeat
```

Memory

Key Insight

Forward calculates; backward corrects.

38. Underfitting

Model is **too simple**.

```
High training error
High test error
```

Example:

Trying to fit a complex nonlinear relationship with an overly simple linear model.

Symptoms

- high bias
- poor training performance
- poor test performance

39. Overfitting

Model learns training data too specifically.

```
Low training error
High test error
```

It captures:

noise
random fluctuations
irrelevant details
instead of general patterns.

Underfitting vs Overfitting

	Underfitting	Overfitting
Model	Too simple	Too complex
Training error	High	Low
Test error	High	High
Generalization	Poor	Poor
Bias	High	Low
Variance	Usually low	High

Solution to underfitting

increase model capacity
better features
train longer
reduce excessive regularization

Solution to overfitting

more training data
regularization
simpler model
feature selection
early stopping
cross-validation
data augmentation where applicable

COMPLETE AI ALGORITHM MAP

This is the map I want you to remember rather than memorizing disconnected algorithms:



FORMULA SHEET

Search

A*

\$\$

$$f(n)=g(n)+h(n)$$

\$\$

Greedy

\$\$

$$f(n)=h(n)$$

\$\$

Probability

\$\$

$$P(A|B)=\frac{P(A,B)}{P(B)}$$

\$\$

Bayes

\$\$

$$P(H|E)=$$

$$\frac{P(E|H)P(H)}{P(E)}$$

$\{P(E)\}$

\$\$

Complement

\$\$

$$P(\neg A) = 1 - P(A)$$

\$\$

Bayesian Network

\$\$

$$P(X_1, \dots, X_n)$$

=

$$\prod_i P(X_i | \text{Parents}(X_i))$$

\$\$

Neural Network

\$\$

$$z = \sum_i w_{ix} x_i + b$$

\$\$

\$\$

$$y = f(z)$$

\$\$

Gradient descent

\$\$

$$w' = w - \eta \nabla L$$

\$\$

EXAM-CRITICAL TOPICS



M1

Intelligent agent

Rationality

PEAS

Types of agents

Environment properties

Agent function vs agent program

Evolution of AI

M2

BFS

DFS

UCS

A*

Heuristics

Hill climbing

Simulated annealing

Minimax

Alpha-beta pruning

M3

Propositional logic

Truth tables

FOL

Quantifiers

Knowledge-based agents

Forward chaining

Backward chaining

Resolution

M4

Planning problem

STRIPS

Goal-stack planning

Planning types

Probability basics

Bayes theorem

Bayesian networks

M5

Types of learning

Induction

Formal learning concepts

Neural networks

Gradient descent

Backpropagation

Underfitting vs overfitting



ACTIVE-RECALL QUESTION BANK

2 marks

Define AI.

Define intelligent agent.

What are sensors and actuators?

What is rationality?

Define PEAS.

What is an agent function?

What is an agent program?

5 marks

Explain types of intelligent agents.

Explain environment types with examples.

Explain PEAS with an example.

Explain evolution of AI.

Explain rational-agent concept.

10 marks

Explain intelligent agents, PEAS and environment types in detail.

Explain different types of intelligent agents with diagrams.

Explain rationality and performance measures.

Discuss evolution and applications of AI.

These match the question structure in your uploaded Module-1 study material.

M2 QUESTIONS

Define state-space search.

Compare BFS and DFS.

Explain UCS.

What is a heuristic?

Explain A*.

What is an admissible heuristic?

Explain hill climbing and its limitations.

Explain simulated annealing.

Explain minimax.

Explain alpha-beta pruning.

Compare minimax and alpha-beta.

Solve a search tree using A*.

Solve a game tree using minimax.

Identify nodes pruned using alpha-beta.

M3 QUESTIONS

What is a knowledge-based agent?
Explain propositional logic.
Construct truth tables.
Convert propositions into logical form.
Explain FOL.
Explain universal/existential quantifiers.
Translate English statements into FOL.
Explain forward chaining.
Explain backward chaining.
Solve a resolution problem.
Compare forward and backward chaining.
Explain Wumpus-world style logical reasoning.

M4 QUESTIONS

Define planning.
What are the components of a planning problem?
Explain STRIPS.
Explain goal-stack planning.
Compare state-space and partial-order planning.
Why is uncertainty important in AI?
Derive Bayes theorem.
Solve a Bayesian inference problem.
Explain Bayesian networks.
Construct a Bayesian network.
Calculate a joint probability using a Bayesian network.

M5 QUESTIONS

Define machine learning.
Explain rote learning.
Explain learning by taking advice.
Explain learning from examples.
What is induction?
Deduction vs induction.
What is hypothesis space?
Explain generalization.
Explain neural-network learning.
Explain gradient descent.
Explain backpropagation.

Underfitting vs overfitting.

Explain methods to reduce overfitting.

NEUROSCIENCE-BASED STUDY SYSTEM FOR AI

For AI, don't memorize algorithms as paragraphs.

Use **five layers of encoding**:

Layer 1 — Meaning

Understand what problem the algorithm solves.

Layer 2 — Visual

Draw it.

Example:

```
A*
g + h
```

Layer 3 — Compression

Reduce it to:

Key Insight

$A^* = \text{cost so far} + \text{estimated cost}$

Layer 4 — Retrieval

Close the notes and reconstruct it.

Layer 5 — Application

Solve a new problem.

The most important technique: CONTRAST

Your brain remembers differences better than isolated facts.

So memorize:

```
BFS vs DFS
Greedy vs A*
Hill Climbing vs Simulated Annealing
Forward vs Backward Chaining
Propositional vs FOL
Bayes vs Bayesian Network
Rote vs Inductive Learning
Underfitting vs Overfitting
Minimax vs Alpha-Beta
```

SPACED-RECALL PLAN

After learning one topic:

```
0 min    → Learn
10 min   → Recall
1 day    → Recall
3 days   → Recall
7 days   → Recall
14 days  → Recall
30 days  → Recall
```

But don't reread.

Recall first.

If you cannot reconstruct:

Key Insight

look → correct → close → recall again.

FINAL 38-TOPIC CHECKLIST

M1 — 7/7

- ☐ What is AI?
- ☐ Evolution
- ☐ Intelligent Agents
- ☐ Rationality
- ☐ Environments
- ☐ Agent Structure
- ☐ Applications

M2 — 6/6

- ☐ Search-Based Problem Solving
- ☐ State Space Search
- ☐ Heuristic Search
- ☐ Local Search
- ☐ Complex Environments
- ☐ Game Tree Search

M3 — 9/9

- ☐ Knowledge-Based Agents
- ☐ Propositional Logic
- ☐ Propositional → Predicate Logic

- ☐ Propositional Logic Agents
- ☐ FOPL
- ☐ FOL Knowledge Representation
- ☐ Forward Chaining
- ☐ Backward Chaining
- ☐ Resolution

M4 — 7/7

- ☐ Planning
- ☐ Planning Components
- ☐ Planning Types
- ☐ Goal Stack Planning
- ☐ Uncertainty
- ☐ Bayesian Inference
- ☐ Probabilistic Reasoning

M5 — 9/9

- ☐ Learning
- ☐ Rote Learning
- ☐ Learning by Advice
- ☐ Learning from Examples
- ☐ Induction
- ☐ Formal Learning Theory
- ☐ Neural Net Learning
- ☐ Underfitting
- ☐ Overfitting

38/38 topics covered.

One correction to the earlier DMCT approach

For **AI**, I would **not add random modern ML/deep-learning topics into your exam syllabus** just because they're related to AI. Your official CS24307 syllabus ends its learning module at neural-net learning, underfitting and overfitting.

So for exam preparation, this **38-topic boundary is the one to master first**. The extra algorithms I included—such as genetic algorithms—are only contextual enrichment, **not additional syllabus requirements**.